

U-Boot memory allocation

- Many commands in U-Boot loading data into memory, or using data from memory, expect a RAM address as argument
- No built-in memory allocation mechanism → up to the user to know usable memory areas to load/use data
- Use the output of `bdinfo` to know the start address and size of RAM
- Avoid the end of the RAM, which is used by the U-Boot code and dynamic memory allocations
- Not the best part of the U-Boot design, sadly



U-Boot memory manipulation commands

- Commands to inspect or modify any memory location, useful for debugging, poking into hardware registers, etc.
- Addresses manipulated in U-Boot are directly physical addresses
- Memory display `md [.b, .w, .l, .q] address [# of objects]`
- Memory write
  `mw [.b, .w, .l, .q] address value [count]`
- Memory modify (modify memory contents interactively starting from address)
  `mm [.b, .w, .l, .q] address`



U-Boot raw storage commands

U-Boot can manipulate raw storage devices:

NAND flash

- `nand info`
- `nand read <addr> <off|partition> <size>`
- `nand erase [<off> [<size>]]`
- `nand write <addr> <off|partition> <size>`
- More: `help nand`

MMC

- `mmc info`
- `mmc read <addr> <blk#> <cnt>`
- `mmc write <addr> <blk#> <cnt>`
- `mmc part` to show partition table
- `mmc dev` to show/set current MMC device
- More: `help mmc`

USB storage

- `usb info`
- `usb read <addr> <blk#> <cnt>`
- `usb write <addr> <blk#> <cnt>`
- `usb part`
- `usb dev`
- More: `help usb`

Note: `<addr>` are addresses in RAM where data is stored



U-Boot commands example

List partitions on MMC

```
STM32MP> mmc part
Partition Map for MMC device 0  --   Partition Type:
EFI

Part    Start LBA       End LBA         Name
        Attributes
        Type GUID
        Partition GUID
  1     0x00000022      0x000001d3      "fsbl1"
        attrs:  0x0000000000000000
        type:   0fc63daf-8483-4772-8e79-3d69d8477de4
        type:   linux
        guid:   72c63477-c475-4cf7-988e-b763bce4604e
  2     0x000001d4      0x00000385      "fsbl2"
        attrs:  0x0000000000000000
        type:   0fc63daf-8483-4772-8e79-3d69d8477de4
        type:   linux
        guid:   66d616db-de56-4a1e-9b13-9b1a5a6e360f
  3     0x00000386      0x00001385      "fip"
        attrs:  0x0000000000000000
        type:   0fc63daf-8483-4772-8e79-3d69d8477de4
        type:   linux
        guid:   6251ecf7-d985-4d81-a396-7a6b6fab8b7c
  [...]
```

Read block 0x22 from MMC to RAM 0xc0000000

```
STM32MP> mmc read c0000000 22 1
MMC read: dev #0, block #34, count 1 ... 1 blocks read: OK
```

Dump memory at 0xc00000000

```
STM32MP> md c0000000
c0000000: 324d5453 00000000 00000000 00000000  STM2............
c0000010: 00000000 00000000 00000000 00000000  ................
c0000020: 00000000 00000000 00000000 00000000  ................
c0000030: 00000000 00000000 00000000 00000000  ................
```



U-Boot filesystem storage commands

- U-Boot has support for many filesystems
  - The exact list of supported filesystems depends on the U-Boot configuration
- Per-filesystem commands
  - FAT: `fatinfo`, `fatls`, `fatsize`, `fatload`, `fatwrite`
  - ext2/3/4: `ext2ls`, `ext4ls`, `ext2load`, `ext4load`, `ext4size`, `ext4write`
  - Squashfs: `sqfsls`, `sqfsload`
- "New" generic commands, working for all filesystem types
  - Load a file:
    `load <interface> [<dev[:part]> [<addr> [<filename> [bytes [pos]]]]]`
  - List files: `ls <interface> [<dev[:part]> [directory]]`
  - Get the size of a file: `size <interface> <dev[:part]> <filename>`
    (result stored in `filesize` environment variable)
  - `interface`: `mmc`, `usb`
  - `dev`: device number, `0` for first device, `1` for second device
  - `part`: partition number



U-Boot filesystem command example

List files

```
STM32MP> ls mmc 0:4
<DIR>       1024 .
<DIR>       1024 ..
<DIR>      12288 lost+found
<DIR>       2048 bin
<DIR>       1024 boot
<DIR>       1024 dev
<DIR>       1024 etc
[...]

STM32MP> ls mmc 0:4 /etc
<DIR>       1024 .
<DIR>       1024 ..
             209 asound.conf
<DIR>       1024 fonts
             334 fstab
             347 group
[...]
```

Load file

```
STM32MP> load mmc 0:4 c0000000 /etc/fstab
334 bytes read in 143 ms (2 KiB/s)
```

Show file contents

```
STM32MP> md c0000000
c0000000: 663c2023 20656c69 74737973 093e6d65  #<filesystem>.
c0000010: 756f6d3c 7020746e 3c093e74 65707974  <mountpt>.\<type
c0000020: 6f3c093e 6f697470 093e736e 6d75643c  >.<options>.\<dum
c0000030: 3c093e70 73736170 642f0a3e 722f7665  p>.<pass>./dev/r
c0000040: 09746f6f 6509092f 09327478 6e2c7772  oot./..ext2.rw,n
[...]
```



U-Boot networking

- Environment variables
  - `ethaddr`: MAC address
  - `ipaddr`: IP address of the board
  - `serverip`: IP address of the server for network related commands
- Important commands
  - `ping`: ping a destination machine. Note: U-Boot is not an operating system with multitasking/interrupts, so ping from another machine to U-Boot cannot work.
  - `tftp`: load a file using the TFTP protocol
  - `dhcp`: get an IP address using DHCP



TFTP

- Network transfer from the development workstation to U-Boot on the target takes place through TFTP
  - *Trivial File Transfer Protocol*
  - Somewhat similar to FTP, but without authentication and over UDP
- A TFTP server is needed on the development workstation
  - `sudo apt install tftpd-hpa`
  - All files in `/srv/tftp` are then visible through TFTP
  - A TFTP client is available in the `tftp-hpa` package, for testing
- A TFTP client is integrated into U-Boot
  - Configure the `ipaddr`, `serverip`, and `ethaddr` environment variables
  - Use `tftp <address> <filename>` to load file contents to the specified RAM address
  - Example: `tftp 0x21000000 zImage`



Scripts in environment variables

- Environment variables can contain small scripts, to execute several commands and test the results of commands.
  - Useful to automate booting or upgrade processes
  - Several commands can be chained using the `;` operator
  - Tests can be done using `if command ; then ... ; else ... ; fi`
  - Scripts are executed using `run <variable-name>`
  - You can reference other variables using `$variable-name`
- Examples
  - ```
    setenv bootcmd tftp 0x21000000 zImage; tftp 0x22000000 dtb;
        bootz 0x21000000 - 0x22000000'
    ```
  - ```
    setenv mmc-boot if fatload mmc 0 80000000 boot.ini; then source;
        else if fatload mmc 0 80000000 zImage; then run mmc-do-boot; fi; fi'
    ```



U-Boot booting commands

- Commands to boot a Linux kernel image
  - `bootz` → boot a compressed ARM32 `zImage`
  - `booti` → boot an uncompressed ARM64 or RISC-V `Image`
  - `bootm` → boot a kernel image with legacy U-Boot headers
  - `zboot` → boot a compressed x86 `bzImage`
- `bootz [addr [initrd[:size]] [fdt]]`
  - `addr`: address of the kernel image in RAM
  - `initrd`: address of the *initrd* or *initramfs*, if any. Otherwise, must pass `-`
  - `fdt`: address of the *Device Tree* passed to the Linux kernel
- Important environment variables
  - `bootcmd`: list of commands executed automatically by U-Boot after the count down
  - `bootargs`: Linux kernel command line



U-Boot booting example

Load kernel image and Device Tree

```
STM32MP> ls mmc 0:4 /boot
<DIR>       1024 .
<DIR>       1024 ..
          117969 stm32mp157c-dk2.dtb
         7538376 zImage

STM32MP> load mmc 0:4 c2000000 /boot/zImage
7538376 bytes read in 463 ms (15.5 MiB/s)

STM32MP> load mmc 0:4 c4000000 /boot/stm32mp157c-dk2.dtb
117969 bytes read in 148 ms (778.3 KiB/s)
```

Set kernel command line and boot

```
STM32MP> setenv bootargs root=/dev/mmcblk0p4 rootwait

STM32MP> bootz c2000000 - c4000000
Kernel image @ 0xc2000000 [ 0x000000 - 0x7306c8 ]
#Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   Loading Device Tree to cffe0000, end cffffcd0 ... OK
[...]
```



FIT image

- U-Boot has a concept of **FIT** image
- FIT = *Flat Image Tree*
- Container format that allows to bundle multiple images into one
  - Multiple kernel images
  - Multiple Device Trees
  - Multiple initramfs
  - Any other image: FPGA bitstream, etc.
- Typically useful for secure booting and to ensure binaries don't overlap in memory.
- Interestingly, relies on the *Device Tree Compiler*
  - `.its` file describes the contents of the image
  - Device Tree Compiler compiles it into an `.itb`
- U-Boot can load an `.itb` image and use its different elements
- https://www.thegoodpenguin.co.uk/blog/u-boot-fit-image-overview/



Generic Distro boot (1)

- Each board/platform used to have its own U-Boot environment, with custom variables/commands
- Wish to standardize the behavior of bootloaders, including U-Boot
- *Generic Distro boot* concept
- If enabled, at boot time, U-Boot:
  - Can be asked to locate a bootable partition (`part list` command), as defined by the bootable flag of the partition table
  - With the `sysboot` command, will look for a `/extlinux/extlinux.conf` or `/boot/extlinux/extlinux.conf` file describing how to boot, and will offer a prompt in the console to choose between available configurations.
  - Once a configuration is selected, will load and boot the corresponding kernel, device tree and initramfs images.
  - Example `bootcmd`:
    `part list mmc 0 -bootable bootpart; sysboot mmc 0:$bootpart any`
- https://u-boot.readthedocs.io/en/latest/develop/distro.html



Generic Distro boot (2)

Several environment variables need to be set:

- `kernel_addr_r`: address in RAM to load the kernel image
- `ramdisk_addr_r`: address in RAM to load the initramfs image (if any)
- `fdt_addr_r`: address in RAM to load the DTB (Flattened Device Tree)
- `pxefile_addr_r`: address in RAM to load the configuration file (usually `extlinux.conf`)
- `bootfile`: the path to the configuration file, for example `/boot/extlinux/extlinux.conf`

Example `/boot/extlinux/extlinux.conf`

```
label stm32mp157c-dk2-buildroot
  kernel /boot/zImage
  devicetree /boot/stm32mp157c-dk2.dtb
  append root=/dev/mmcblk0p4 rootwait
```

U-Boot boot log

```
Hit any key to stop autoboot:  0
Boot over mmc0!
switch to partitions #0, OK
mmc0 is current device Scanning mmc 0:4...
Found /boot/extlinux/extlinux.conf
Retrieving file: /boot/extlinux/extlinux.conf
131 bytes read in 143 ms (0 Bytes/s)
1:      stm32mp157c-dk2-buildroot
Retrieving file: /boot/zImage
7538376 bytes read in 462 ms (15.6 MiB/s)
append: root=/dev/mmcblk0p4 rootwait
Retrieving file: /boot/stm32mp157c-dk2.dtb
117969 bytes read in 148 ms (778.3 KiB/s)
Kernel image @ 0xc2000000 [ 0x000000 - 0x7306c8 ]
## Flattened Device Tree blob at c4000000
  Booting using the fdt blob at 0xc4000000
  Loading Device Tree to cffe0000, end cffffcd0 ... OK

Starting kernel ...
```



TF-A: Trusted Firmware



Concept of FIP

- FIP = *Firmware Image Package*
- Concept specific to TF-A
- *packaging format used by TF-A to package firmware images in a single binary*
- Typically used to bundle the BL33, i.e. the U-Boot bootloader that will be loaded by TF-A.
- https://trustedfirmware-a.readthedocs.io/en/latest/getting_started/tools-build.html
- https://wiki.st.com/stm32mpu/wiki/How_to_configure_TF-A_FIP



Configuring TF-A

- TF-A does not use *Kconfig* for configuration
- All the configuration is based on variables passed on the `make` command line
- Most variables are documented in TF-A's getting started



Configure TF-A: important variables

- `CROSS_COMPILE`, cross-compiler prefix
- `ARCH`, CPU architecture: `aarch32` or `aarch64`
- `ARM_ARCH_MAJOR`, `7` for ARMv7, `8` for ARMv8
- `PLAT`, SoC family, any directory name in `plat` that contains `platform.mk`
- `AARCH32_SP`, the Secure Payload, specific to ARMv7. Either OP-TEE or the built-in *SP-MIN* provided by TF-A
- `DTB_FILE_NAME`, path to the Device Tree describing our board
- `BL33`, path to the second stage bootloader, usually U-Boot, to include in the FIP image
- Specific to STM32MP1
  - `BL33_CFG`, path to the U-Boot Device Tree
  - `STM32MP_SDMMC=1`, enable support for SD card/eMMC in TF-A



Building TF-A for STM32MP1

```
$ make CROSS_COMPILE=arm-linux-
       ARM_ARCH_MAJOR=7
       ARCH=aarch32
       PLAT=stm32mp1
       AARCH32_SP=sp_min
       DTB_FILE_NAME=stm32mp157a-dk1.dtb
       BL33=/path/to/u-boot/u-boot-nodtb.bin
       BL33_CFG=/path/to/u-boot/u-boot.dtb
       STM32MP_SDMMC=1
       fip all
```

Build results in `build/stm32mp1/release`. Important files:

- `tf-a-stm32mp157a-dk1.stm32`, TF-A itself
- `fip.bin`, the FIP image, containing U-Boot and other elements



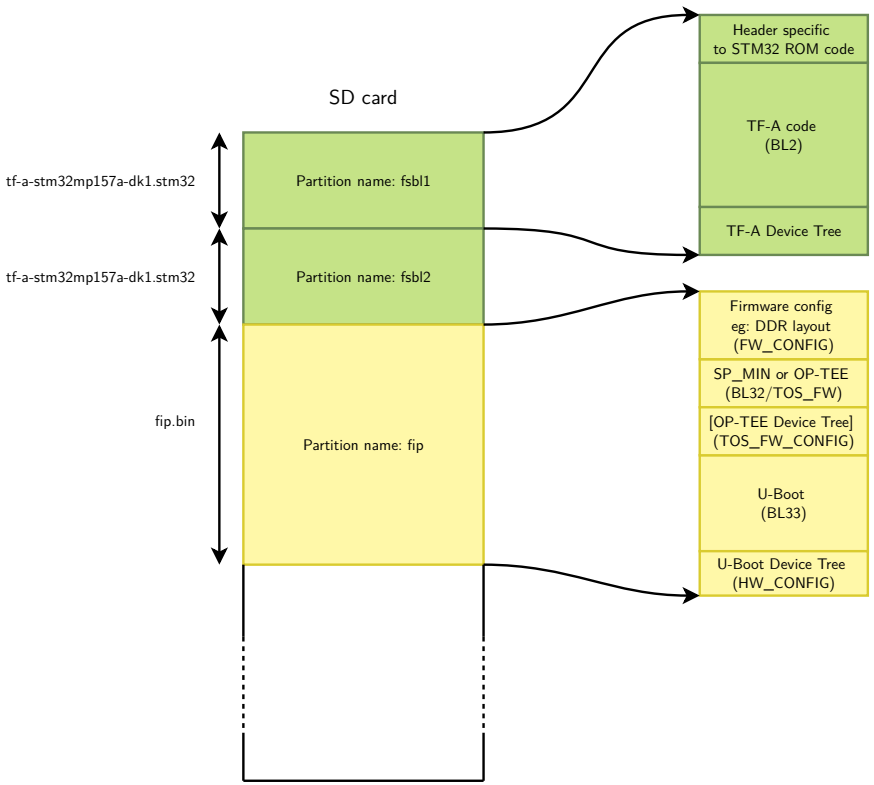
FIP image contents

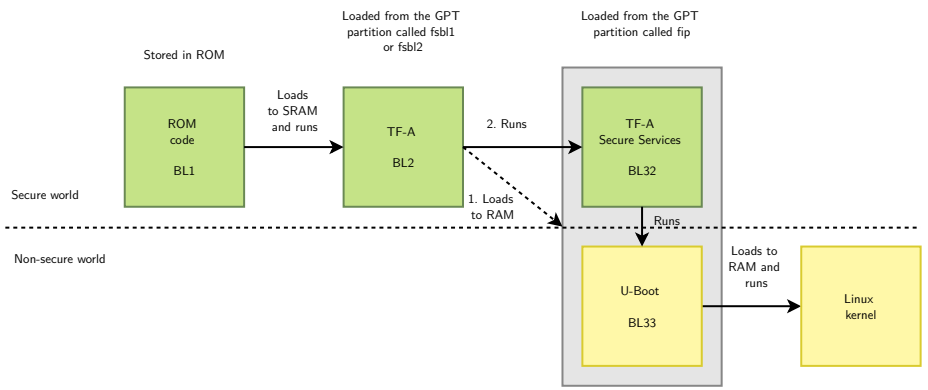
fiptool info

```
$ ./tools/fiptool/fiptool info build/stm32mp1/release/fip.bin
Secure Payload BL32 (Trusted OS): offset=0x100, size=0x8AEC, cmdline="--tos-fw"
Non-Trusted Firmware BL33: offset=0x8BEC, size=0xECE6C, cmdline="--nt-fw"
FW_CONFIG: offset=0xF5A58, size=0x226, cmdline="--fw-config"
HW_CONFIG: offset=0xF5C7E, size=0x16A98, cmdline="--hw-config"
TOS_FW_CONFIG: offset=0x10C716, size=0x3CF6, cmdline="--tos-fw-config"
```



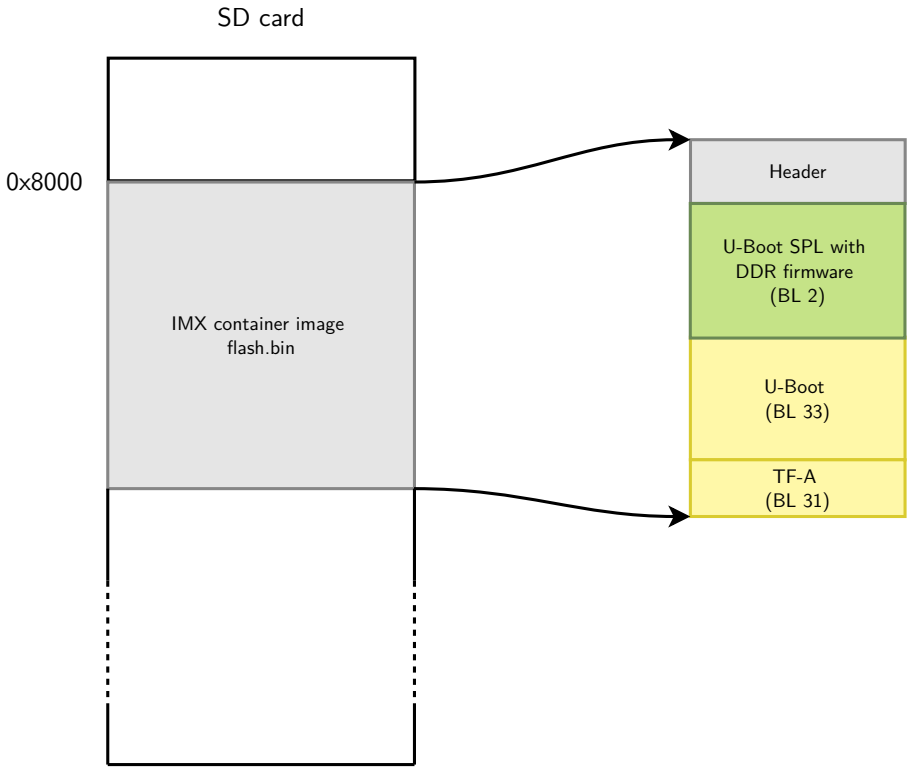
STM32MP1 partition layout

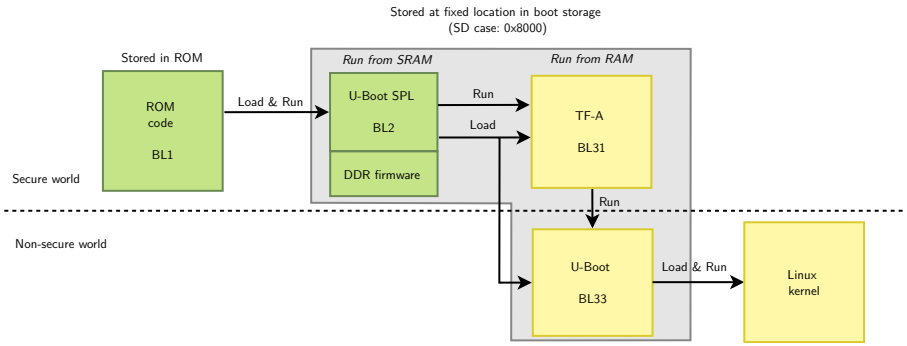




Reminder: STM32MP1's boot sequence

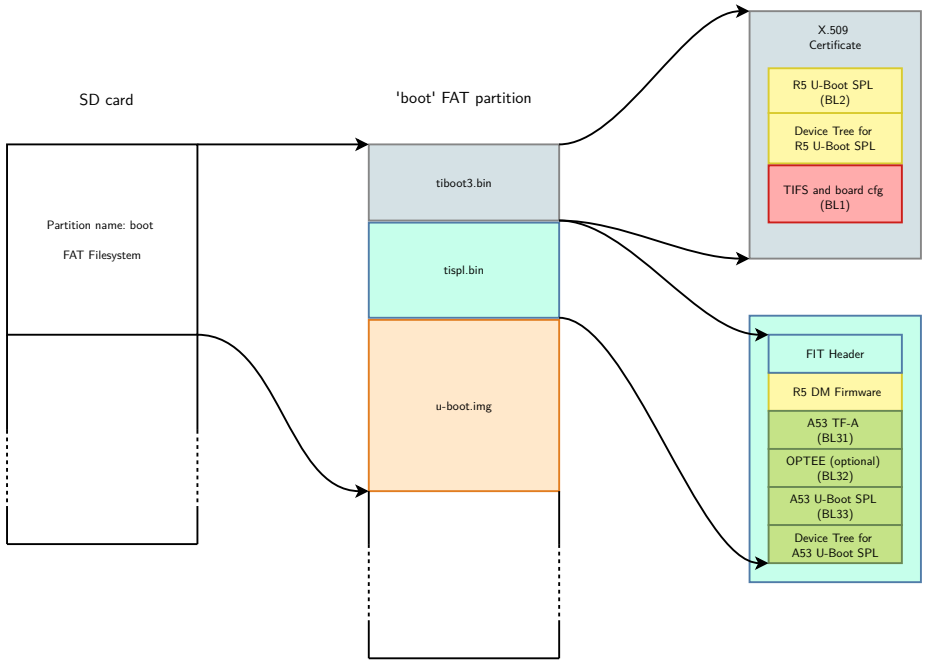
IMX93 partition layout

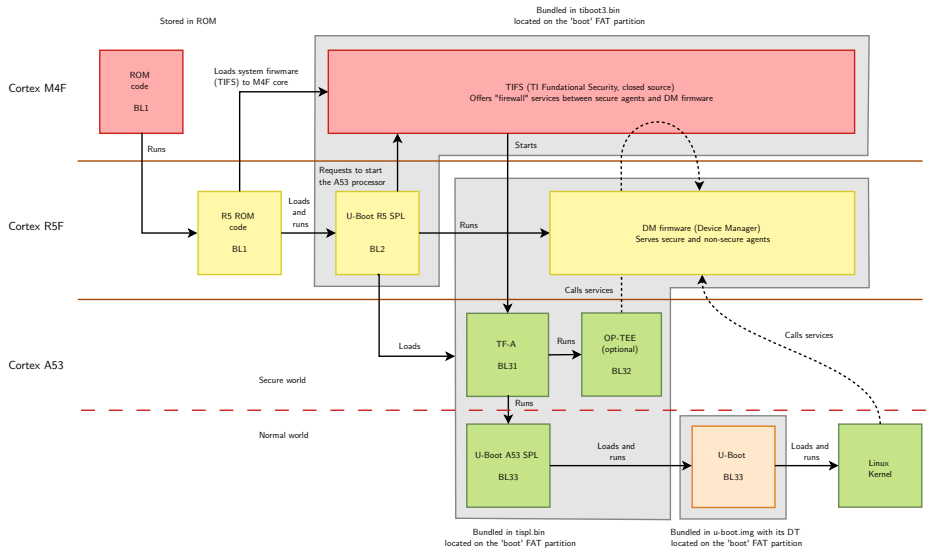




Reminder: IMX93's boot sequence

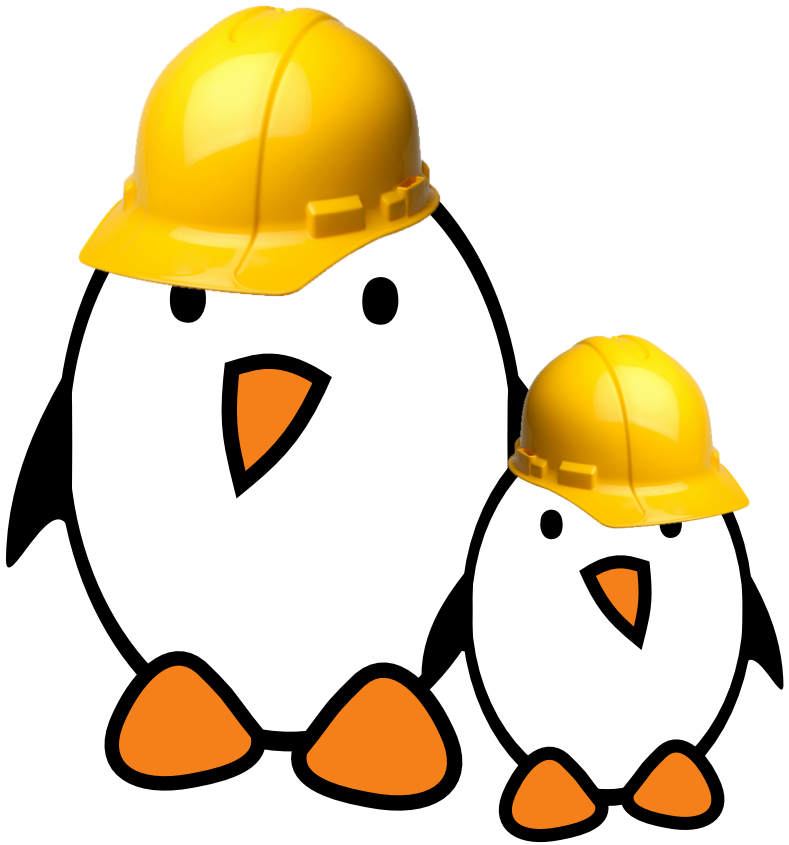
AM62x (BeaglePlay) partition layout





Reminder: AM62x's boot sequence

Practical lab - U-Boot / TF-A



Time to start the practical lab!

- Communicate with the board using a serial console
- Configure, build and install the bootloader stages:
  - *TF-A* and *U-Boot* on STM32MP1 and Beagleplay
  - *U-Boot SPL* and *U-Boot* on BeagleBoneBlack and QEMU
- Learn *U-Boot* commands
- Set up *TFTP* communication with the host